

TaskFlow – Secure Role-Based Project, Task & Developer Workload Management Platform

1. Project Overview

TaskFlow is a web-based **project, task, team, and workload management platform** designed to help organizations manage software-development activities in a centralized and secure environment.

The main idea behind TaskFlow is not simply to create and track tasks. It is to provide **complete visibility of work being performed by different team members**, while ensuring that each user can access only the information and operations appropriate to their role.

In a typical software-development environment, a Team Lead or Project Manager may see only the tasks formally assigned to a developer. However, a developer may also be:

- Mentoring interns
- Guiding junior developers
- Reviewing code
- Helping teammates solve technical issues
- Attending project meetings
- Handling urgent support issues
- Working on multiple projects
- Performing work outside their primary technical domain

These responsibilities may not always appear as formal tickets. As a result, the developer's **actual workload can be significantly higher than the workload visible to the manager**.

TaskFlow addresses this problem by combining:

Task Management + Project Management + Role-Based Access Control + Ownership-Based Security + Workload Transparency + Domain Matching + Analytics + Reporting

The platform provides different interfaces and capabilities based on organizational roles such as:

- **ADMIN**
- **PROJECT MANAGER**
- **TEAM LEAD**
- **DEVELOPER**
- **JUNIOR DEVELOPER**

- **INTERN**

The ultimate objective is to ensure:

The right person gets the right information, the right task, and the right level of responsibility while management gets a realistic view of team workload.

2. Problem Statement

Modern software projects involve multiple people working on different tasks, projects, technologies, and responsibilities.

A conventional task-management system generally focuses on:

Who has been assigned which ticket?

However, this does not always represent the developer's actual workload.

For example, consider a developer working as a Backend Developer.

They may have:

- 3 backend tickets
- 2 interns under their guidance
- 4 code reviews
- 2 technical-support requests
- Daily meetings
- An urgent production issue

The task-management system might show only:

3 assigned tickets

The Team Lead may therefore assume that the developer has sufficient capacity for another task.

This creates a gap between:

Visible Workload

and

Actual Workload

There is another problem when a developer is suddenly assigned work outside their primary technical domain.

For example:

Developer:

Backend

Current Tasks:

Spring Boot API

Database Integration

Authentication

New Ticket:

React UI Bug

The developer now has to move from backend development to frontend development. Such context switching can interrupt work continuity and increase the effort required to complete tasks efficiently.

Therefore, TaskFlow focuses on **workload transparency and domain-aware task assignment**, in addition to conventional project and task management.

3. Main Objective of TaskFlow

The primary objective of TaskFlow is:

To develop a secure, role-based project and task management platform that provides transparent visibility into project progress, individual workload, task ownership, team responsibilities, and technical-domain assignments.

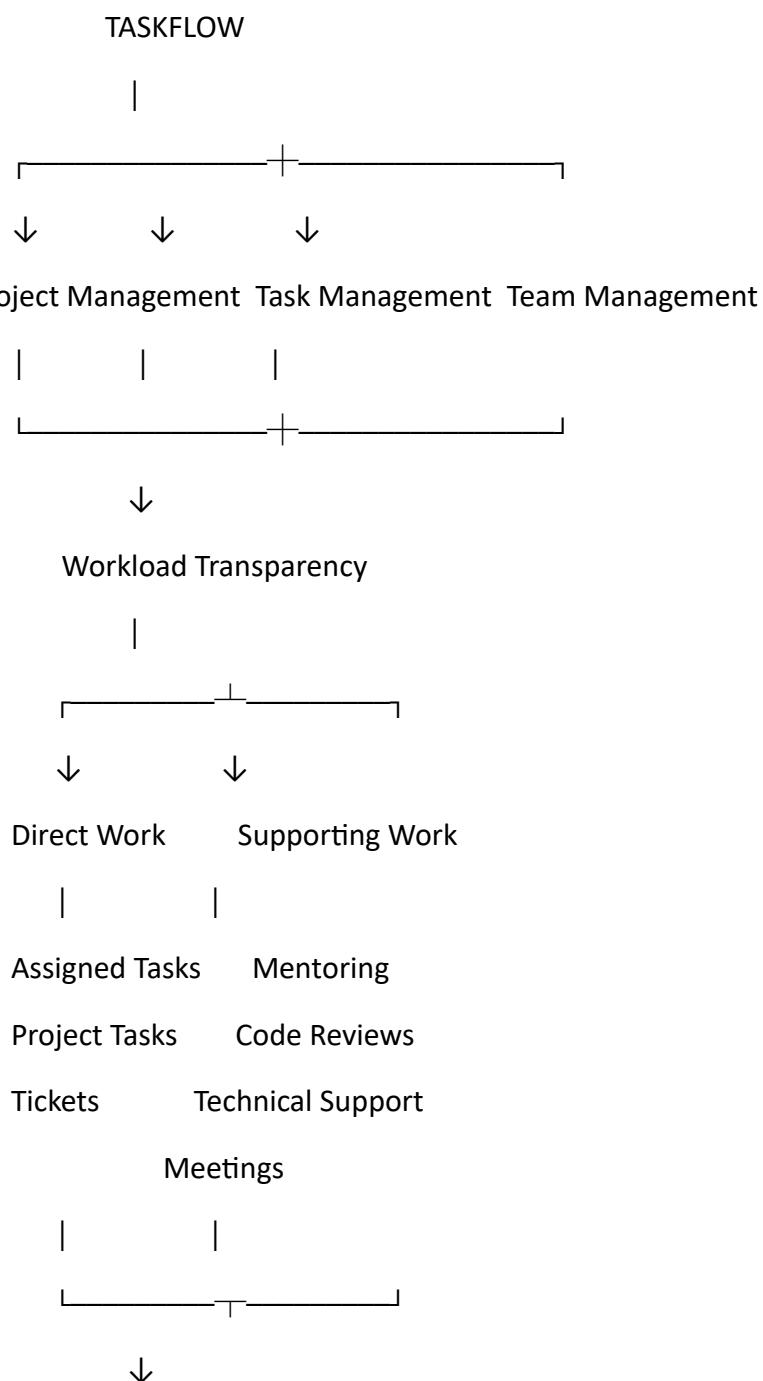
The system aims to help organizations:

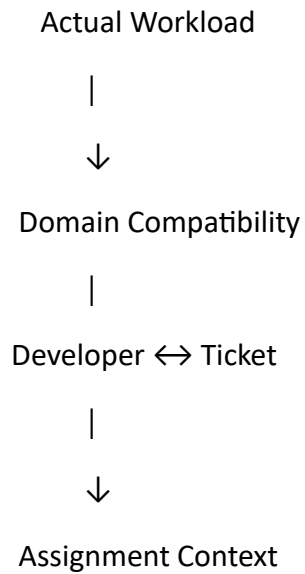
1. Centralize project and task management.
2. Assign tasks according to responsibilities and technical domains.
3. Enforce role-based access control.
4. Enforce task ownership restrictions.
5. Make hidden developer workload visible.
6. Account for mentoring and support responsibilities.

7. Identify potential domain mismatches before task assignment.
8. Monitor project and team progress.
9. Provide analytics and reports.
10. Improve workload distribution and management decision-making.

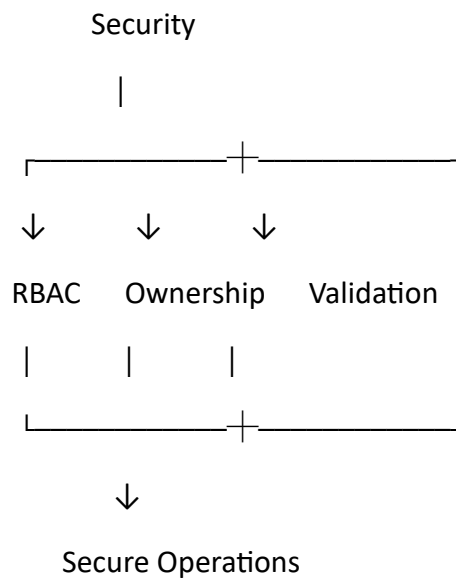
4. Core Concept of TaskFlow

TaskFlow can be viewed as five major layers:





Along with this, security operates across the entire system:



5. User Roles

TaskFlow supports different levels of responsibility.

ADMIN

The Admin has the highest level of system access.

The Admin can:

- Manage users
- Manage projects
- Manage tasks

- Manage teams
 - View global analytics
 - View reports
 - Manage system settings
 - Perform authorized CRUD operations
-

PROJECT MANAGER

The Project Manager focuses primarily on project execution.

They can:

- Create and manage projects
 - Create and assign tasks
 - Monitor project progress
 - Monitor team workload
 - View analytics
 - Track deadlines
 - Generate reports
 - Monitor project performance
-

TEAM LEAD

The Team Lead manages the work of a particular team.

They can:

- View relevant team information
- Create/manage authorized tasks
- Assign work
- Monitor team workload
- Monitor developer availability
- Monitor mentoring responsibilities
- View domain compatibility

- Track project/team progress

This role is particularly important for the **developer workload transparency feature**.

DEVELOPER

The Developer can:

- View assigned tasks
- Update authorized tasks
- Update task progress
- View relevant projects
- Record supporting responsibilities
- View personal workload
- View technical-domain assignments
- See upcoming deadlines

The Developer does not receive unrestricted access to global task management.

JUNIOR DEVELOPER

The Junior Developer receives more restricted access.

They can:

- View authorized tasks
 - Update permitted tasks
 - View relevant project information
 - Report progress
 - Collaborate with assigned team members
-

INTERN

The Intern receives limited access.

They can:

- View assigned tasks

- Update permitted task information
 - Submit progress
 - Access relevant project information
-

6. Major Functional Modules

6.1 Authentication Module

Users authenticate into TaskFlow using their credentials.

After authentication, the system determines the user's role.

For example:

Login



Authentication



Identify User



Identify Role



Load Appropriate Permissions



Display Role-Specific Dashboard

Therefore, an Admin does not receive the same interface as an Intern.

7. Role-Based Access Control

One of the major security features of TaskFlow is **RBAC – Role-Based Access Control**.

Instead of allowing every user to perform every operation, permissions are associated with organizational roles.

For example:

Operation	Admin	PM	Team Lead	Developer	Junior Dev	Intern
Create Task	✓	✓	✓	Restricted	Restricted	X
Global Task View	✓	✓	✓	X	X	X
Assigned Task View	✓	✓	✓	✓	✓	✓
Update Own/Assigned Task	✓	✓	✓	✓	✓	Restricted
Delete Task	✓	✓	Restricted	X	X	X
Analytics	✓	✓	✓	Personal	Restricted	Restricted
Team Management	✓	✓	✓	X	X	X

The exact permission matrix can be configured according to organizational requirements.

Important principle

Unauthorized functionality should not merely be disabled in the frontend.

The **backend must enforce authorization**.

Therefore:

Frontend Security

+

Backend Security

=

Actual Security

8. Task Management

TaskFlow provides complete task lifecycle management.

A task can contain:

Task ID

Task Name

Description

Project

Assigned Developer

Technical Domain

Priority

Status

Start Time

End Time

Progress

Example:

Task: Implement Login API

Project: TaskFlow

Assigned To: Arun

Domain: Backend

Priority: High

Status: In Progress

Progress: 65%

Start: 20 Sep

Deadline: 27 Sep

The system can track the task from creation to completion.

9. Project Management

Project Managers and authorized users can create and monitor projects.

A project can contain:

- Project name
- Description
- Start date
- End date
- Team members
- Tasks

- Milestones
- Project progress
- Status
- Deadlines

For example:

Project: E-Commerce Platform

Progress: 72%



This allows management to understand the overall project status.

10. Developer Workload Transparency

This is one of the **key real-world problems addressed by TaskFlow**.

Traditional task tracking often considers only formally assigned tickets.

TaskFlow expands workload visibility by considering different types of work.

Direct workload

Examples:

- Assigned tickets
- Feature development
- Bug fixing
- Testing
- Project tasks

Supporting workload

Examples:

- Mentoring interns
- Helping junior developers
- Code reviews
- Technical support
- Team meetings
- Knowledge transfer
- Troubleshooting

Therefore:

Actual Developer Workload

=

Direct Work

+

Supporting Responsibilities

11. Example of Hidden Workload

Suppose:

Developer: Arun

Domain: Backend

His visible tasks:

API Development 25%

Database Work 15%

Authentication 15%

Visible workload 55%

But he is also mentoring two interns:

Intern Mentoring 15%

Code Reviews 8%

Technical Support 7%

Meetings 5%

Therefore:

Visible ticket workload = 55%

Additional responsibilities = 35%

Actual workload = 90%

A conventional system might display:

55% workload

TaskFlow can display:

90% effective workload

This provides the Team Lead with a more realistic picture.

12. Developer Mentoring Workload

This is particularly important in organizations where experienced developers supervise less experienced members.

For example:

Arun – Senior Developer

Direct Tasks:

3

Interns Supported:

2

Junior Developers Supported:

1

Code Reviews:

5

Technical Support:

3

The system can therefore show:

Direct Workload 55%

Mentoring Work 18%

Code Reviews 8%

Support 7%

Meetings 5%

Effective Workload 93%

This prevents mentoring responsibilities from becoming **invisible work**.

13. Domain-Based Task Management

TaskFlow can associate developers and tickets with technical domains.

Example domains:

- Frontend
- Backend
- Database
- DevOps
- Testing
- Mobile
- UI/UX
- AI/ML

- Cybersecurity

A developer can have:

Primary Domain

Backend

Secondary Domains

Database

DevOps

A ticket can also contain a domain:

Ticket:

Fix React Dashboard

Domain:

Frontend

TaskFlow can compare:

Developer Domain



Ticket Domain



Compatibility

14. Domain Mismatch Problem

Consider:

Developer:

Backend

Current Work:

Spring Boot API

MySQL Integration

Authentication

Current Workload:

82%

New Ticket:

React Dashboard UI Bug

Ticket Domain:

Frontend

The system can highlight:

⚠ High Workload

⚠ Different Technical Domain

⚠ Potential Context Switching

This does **not mean that the developer cannot handle the ticket.**

Instead, TaskFlow provides information to the Team Lead before assigning it.

The Team Lead can then consider:

- Current workload
- Technical domain
- Existing deadlines
- Developer availability
- Required skills
- Project priority

15. Assignment Intelligence

When creating a ticket, the Team Lead can see relevant information about potential assignees.

Example:

Create Ticket

Ticket:

Payment API Bug

Domain:

Backend

Priority:

High

Potential developers:

Arun

Domain: Backend

Workload: 86%

Domain Match: ✓

Status: High Workload

Karthik

Domain: Backend

Workload: 48%

Domain Match: ✓

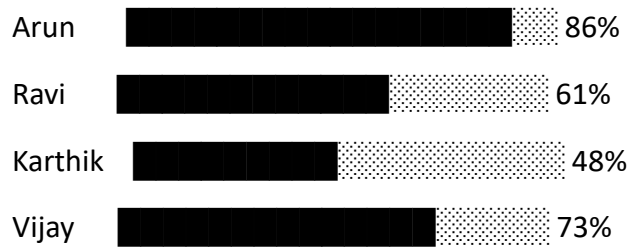
Status: Available

The system provides **decision-support information** rather than automatically making the management decision.

16. Workload Visualization

TaskFlow can provide workload charts such as:

Team Workload



And workload categories:

Direct Tasks	55%
Mentoring	18%
Code Reviews	8%
Meetings	5%
Support	7%
Other	3%

This allows managers to identify workload distribution more clearly.

17. Project and Task Analytics

TaskFlow provides analytics to help managers understand project performance.

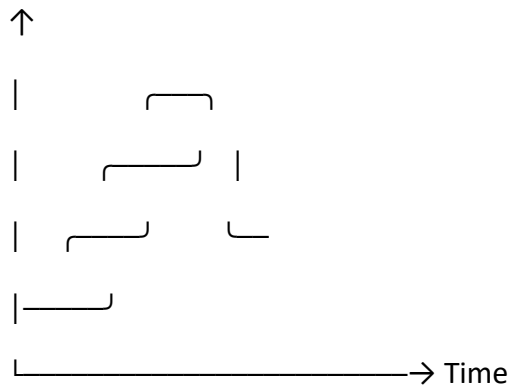
Possible metrics include:

- Total tasks
- Completed tasks
- Pending tasks
- In-progress tasks
- Overdue tasks
- Completion rate
- Average task duration
- Project progress
- Developer workload
- Team productivity
- Domain distribution
- Deadline trends

For example:

Task Completion Trend

Tasks



18. Calendar and Deadline Management

TaskFlow includes a calendar for:

- Task deadlines
- Project deadlines
- Meetings
- Sprint planning
- Reviews
- Milestones

Example:

September

Mon Tue Wed Thu Fri

	1	2	3	
4	5	6	7	8
11	12	13	14	15
18	19	20	21	22
25	26	27	28	29

27 → Backend API Deadline

28 → Sprint Review

29 → Project Milestone

This connects task management with time management.

19. Reports

TaskFlow can generate reports based on:

Task Report

- Total tasks
- Completed
- Pending
- Overdue
- Priority distribution

Project Report

- Project progress
- Milestones
- Task completion
- Deadlines

Team Report

- Team workload
- Completed tasks
- Pending tasks
- Work distribution

Developer Workload Report

- Assigned work
- Mentoring work
- Support work

- Code reviews
 - Current workload
 - Domain distribution
-

20. Data Validation

TaskFlow also validates incoming task data.

For example, mandatory fields such as:

- Task name
- Start time
- Other required fields

must be valid before the task is stored.

The backend uses request validation so invalid requests produce an appropriate:

400 BAD REQUEST

rather than being silently accepted.

This improves both data quality and API reliability.

21. Ownership-Based Security

TaskFlow does not rely only on roles.

It can also consider **who owns or is assigned to the task**.

For example:

Developer A



Assigned Task



Can access permitted operations

But:

Developer A



Task assigned to Developer B



Unauthorized

This provides a second layer of authorization.

Therefore:

Access = Role + Ownership + Operation

rather than simply:

Access = Role

22. Protection Against Resource Enumeration

Suppose a developer tries to access:

GET /tasks/getById/500

where Task 500 belongs to another user.

Instead of exposing information about the protected resource, the backend can return:

404 NOT FOUND

for unauthorized access to an existing protected task.

This helps avoid revealing whether a particular task ID exists.

Therefore, TaskFlow does not only protect the frontend interface; it also protects API-level access.

23. Error Handling

The system handles different types of errors.

Error Meaning

400 Invalid request/data

401 Authentication required

403 User does not have permission

404 Resource unavailable/not accessible

Error Meaning

500 Internal server error

This provides predictable API behavior.

24. Real-World Problems Addressed by TaskFlow

Problem 1 – Fragmented Project Management

Projects, tasks, deadlines and team activities may be tracked separately.

TaskFlow solution

A centralized platform connects:

Projects



Tasks



Team Members



Deadlines



Analytics



Reports

Problem 2 – Same Interface for Different Roles

An Admin, Team Lead, Developer and Intern do not have the same responsibilities.

TaskFlow solution

Role-specific interfaces and permissions ensure that each user sees relevant functionality.

Problem 3 – UI-Only Security

Simply hiding buttons in the frontend is not sufficient.

TaskFlow solution

Authorization is enforced at the backend/API level.

Problem 4 – Unauthorized Task Access

A user may attempt to access another user's task.

TaskFlow solution

Task ownership and role-based authorization are checked before operations are performed.

Problem 5 – Hidden Developer Workload

Developers may have significant responsibilities that are not represented by tickets.

TaskFlow solution

Workload includes:

Assigned Tasks

+

Mentoring

+

Code Reviews

+

Support

+

Meetings

+

Other Responsibilities

Problem 6 – Invisible Mentoring Responsibilities

A developer may be responsible for multiple interns or junior developers.

However, this responsibility may not appear in the project-management dashboard.

TaskFlow solution

Mentoring/support activities become part of workload visibility.

Problem 7 – Overloading Developers

A developer may appear to have available capacity because the manager sees only formal tickets.

TaskFlow solution

TaskFlow calculates and displays a broader workload picture so management can see when a developer is already heavily occupied.

Problem 8 – Domain Mismatch

A Backend Developer may suddenly receive a Frontend ticket.

TaskFlow solution

The system associates:

Developer → Technical Domain

Ticket → Technical Domain

and highlights potential mismatches.

Problem 9 – Excessive Context Switching

Repeatedly moving between unrelated technical areas can interrupt work continuity and increase cognitive overhead.

TaskFlow solution

Domain information is shown alongside workload information when tasks are assigned.

Problem 10 – Poor Workload Distribution

One developer may have 90% workload while another has 40%.

TaskFlow solution

Team workload visualization allows authorized managers to compare workload distribution across the team.

Problem 11 – Lack of Management Visibility

Managers may know the status of tickets but not the broader work being performed by team members.

TaskFlow solution

TaskFlow provides:

- Workload analytics
- Project analytics
- Team analytics
- Developer workload
- Domain distribution
- Supporting responsibilities

Problem 12 – Manual Reporting

Managers may need to manually collect information from multiple sources.

TaskFlow solution

TaskFlow centralizes task, project, team and workload information and provides reports.

Problem 13 – Poor Data Validation

Invalid task information can enter the system.

TaskFlow solution

Backend request validation prevents invalid data from being processed.

25. What Makes TaskFlow Different as a Project Concept

The project should not be presented as:

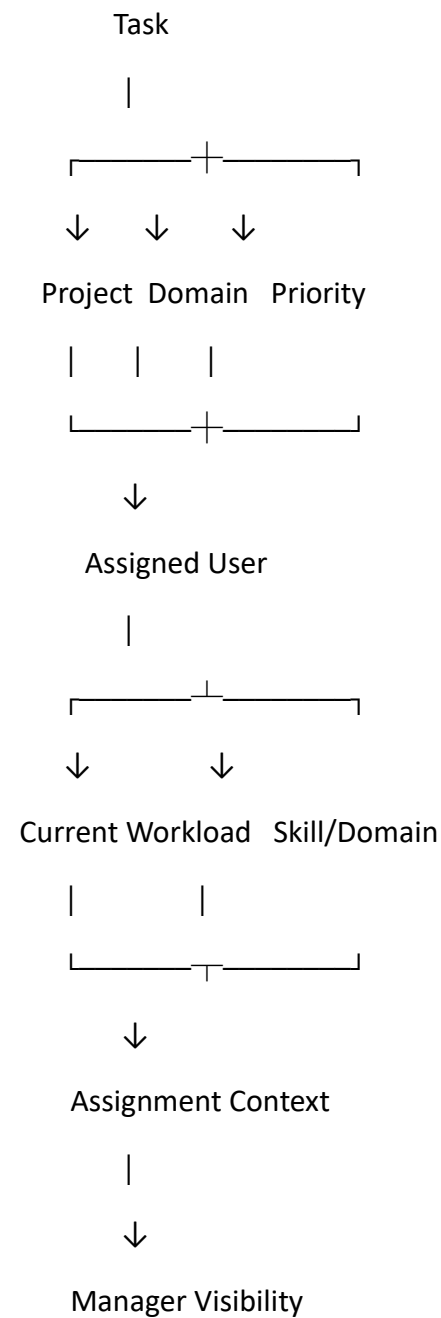
"Another application like Trello/Jira."

A stronger description is:

TaskFlow is a focused role-aware project management and workload transparency platform that integrates task management with organizational roles, task ownership,

developer mentoring responsibilities, workload visibility, and technical-domain awareness.

The important idea is the connection between these components.



26. End-to-End Workflow

A typical TaskFlow workflow can be:

Admin creates users



Roles assigned



Project Manager creates project



Team is associated with project



Team Lead manages team tasks



Developer domains are defined



Tasks/Tickets are created



Ticket domain is identified



Potential assignees are evaluated



Current workload is displayed



Team Lead assigns task



Developer performs task



Developer updates progress



Mentoring/support work is recorded



Effective workload is updated



Project analytics updated



Reports generated

27. Example Real-World Scenario

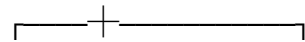
Consider a software company developing an e-commerce application.

Team

Project Manager



Team Lead



Backend Frontend QA

Dev Dev Dev



└─ Intern 1

└─ Intern 2

The Backend Developer currently has:

API Development 25%

Database Integration 15%

Authentication 15%

Intern Mentoring 15%

Code Reviews 8%

Technical Support 7%

Meetings 5%

Effective Workload 90%

Now a new ticket arrives:

Fix React checkout UI

The ticket belongs to:

Domain: Frontend

TaskFlow displays:

Developer:

Backend Developer

Current Workload:

90%

Ticket Domain:

Frontend

Primary Domain:

Backend

⚠ High Workload

⚠ Domain Mismatch

The Team Lead now has complete context before assigning the ticket.

This is the key value of TaskFlow:

The system makes hidden work visible before additional work is assigned.

28. Expected Benefits

For Project Managers

- Better project visibility
- Better deadline tracking
- Better resource visibility

- Centralized reports
- Project performance analytics

For Team Leads

- Better workload visibility
- Better task assignment context
- Visibility into mentoring responsibilities
- Domain-aware assignment information
- Team performance monitoring

For Developers

- Clear task responsibilities
- Visibility into personal workload
- Recognition of mentoring/support work
- Better prioritization
- Reduced risk of unnoticed overload

For Junior Developers and Interns

- Clear assignment visibility
- Clear responsibilities
- Better collaboration
- Structured reporting of progress

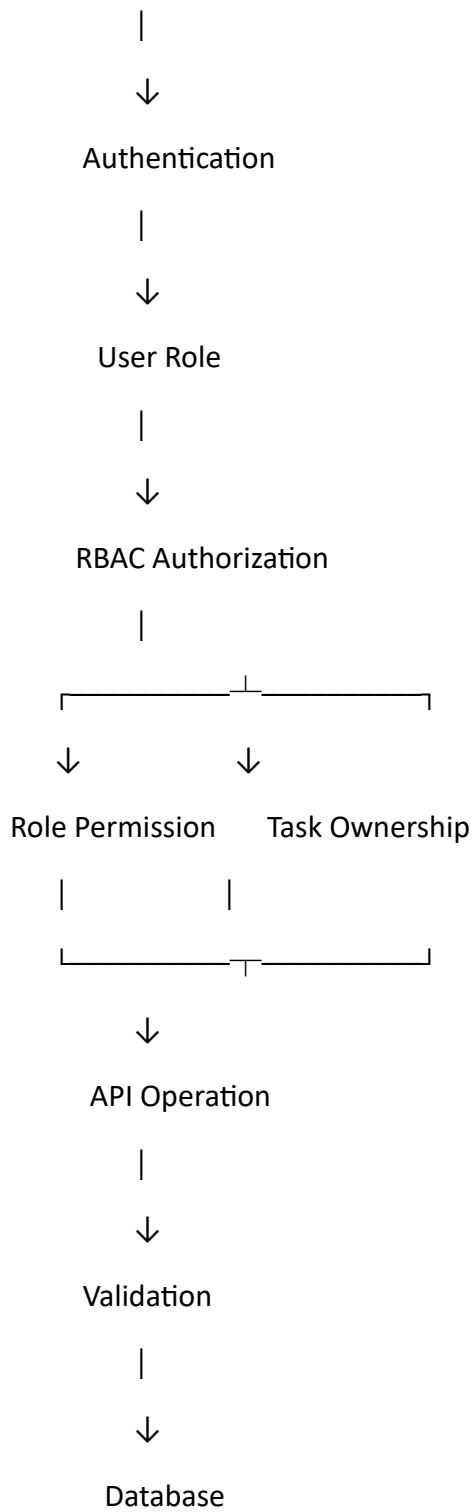
For Administrators

- Centralized user management
- Role-based security
- System-wide visibility
- Controlled access

29. Security Architecture

The security model can be summarized as:

User

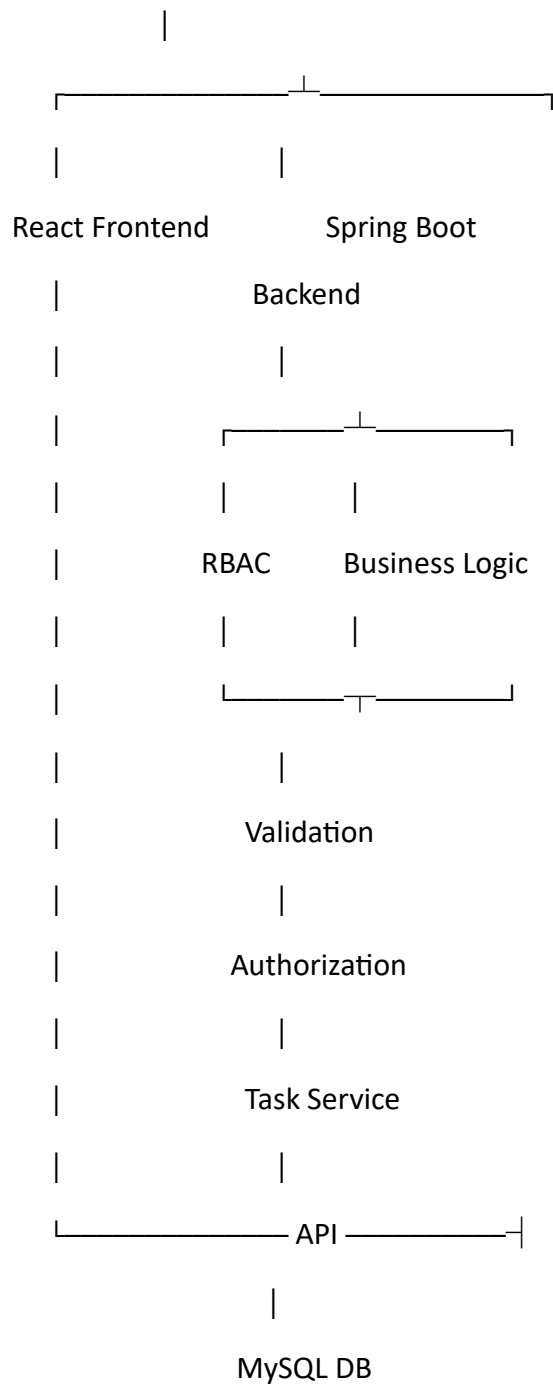


This ensures that security is enforced throughout the application.

30. Overall System Architecture

A high-level architecture can be represented as:

TASKFLOW



The frontend focuses on user experience and role-specific interfaces, while the backend remains the primary enforcement layer for authorization, validation, and business rules.

31. Key Innovation Areas

For your project presentation, I would highlight these **five major areas**:

1. Role-Aware Project Management

Different roles receive different capabilities and interfaces.

2. Ownership-Based Task Security

Access depends not only on role but also on task ownership/assignment.

3. Transparent Developer Workload

The platform considers both formal tasks and supporting responsibilities such as mentoring, reviews and assistance.

4. Domain-Aware Task Assignment

Developer domains and ticket domains are compared to expose potential mismatches.

5. Management Decision Support

Workload, domain, progress and deadlines are brought together so managers have better information when allocating work.

32. Final Project Description

Short version for synopsis

TaskFlow is a secure, role-based project and task management platform designed to centralize project execution, task assignment, team coordination, workload monitoring, analytics and reporting. The system supports multiple organizational roles including Admin, Project Manager, Team Lead, Developer, Junior Developer and Intern, with permissions enforced at the backend using role-based and ownership-based authorization. A key feature of TaskFlow is transparent developer workload management, where the system considers not only assigned tickets but also mentoring, code reviews, technical support and other supporting responsibilities. The platform also associates developers and tickets with technical domains such as Frontend, Backend, DevOps and Testing, enabling Team Leads to identify potential domain mismatches and workload concerns before assigning new work. Through dashboards, analytics, calendars and reports, TaskFlow provides centralized visibility into project progress, team workload and individual responsibilities while maintaining secure access to organizational data.

One-line definition

TaskFlow is a secure, role-aware project management platform that combines task tracking, team coordination, workload transparency and domain-aware task assignment to provide a realistic view of project work and developer capacity.

Core problem solved

TaskFlow addresses the gap between the work that is formally assigned to developers and the work they actually perform by making direct tasks, mentoring responsibilities, support

activities, workload distribution and technical-domain assignments visible to authorized managers and team leads.

This gives your project a much stronger identity: **it is not just about tracking whether a ticket is completed; it is about understanding the actual distribution and nature of work inside a software-development team while enforcing secure, role-appropriate access.**